

Rapport sécurité et hardening - DataVision AI v2.81.0

1. Position de sécurité

La v2.81.0 est un gel de sécurité précédant la v3.0.0. Son objectif est de supprimer l'accès anonyme implicite, durcir l'authentification, consolider la supply chain et documenter les critères de go/no-go.

Important : ce rapport ne certifie pas l'absence absolue de vulnérabilités. Aucun contrôle statique ou test unitaire ne permet une telle garantie. La sécurité production exige également des scans sur l'artefact réellement construit, un pentest indépendant et une revue de l'infrastructure cible.

2. Corrections critiques introduites en v2.81

2.1 Authentification obligatoire

En v2.80, certaines surfaces pouvaient fonctionner en fallback local en absence de headers gouvernés. En v2.81 :

- AUTH_MODE=required est le défaut ;
- datasets, notebooks et assistant rejettent l'accès anonyme ;
- AUTH_MODE=local_dev est explicite et ne peut pas être utilisé selon la sémantique production ;
- le frontend présente un écran d'authentification global.

2.2 Bootstrap protégé

Lorsque l'authentification est requise, la création du premier propriétaire exige BOOTSTRAP_SECRET. Cela réduit le risque de « first user takeover » sur une instance fraîche exposée trop tôt.

2.3 Sessions et tokens

- sessions persistées et révocables ;
- refresh tokens stockés sous forme hashée côté serveur et rotatifs ;
- access tokens de durée limitée ;
- validation de l'état de session à chaque contexte gouverné ;
- tokens navigateur déplacés de localStorage vers sessionStorage.

Le stockage navigateur reste accessible au JavaScript de la page : une XSS réussie pourrait donc encore voler un token. La CSP, l'hygiène des dépendances et les audits réduisent ce risque sans l'annuler. Une architecture cookie HttpOnly pourrait être étudiée ultérieurement si le modèle de déploiement l'exige.

3. Authentification forte

La plateforme supporte :

- mot de passe avec longueur minimale de 15 caractères par défaut pour les nouveaux secrets ;
- hash mémoire-dur scrypt avec sel aléatoire et coût par défaut $N=2^{17}$, $r=8$, $p=1$;
- comparaison constante ;
- limitation des tentatives de connexion ;
- WebAuthn/MFA ;
- OIDC/SSO ;
- SCIM pour provisioning Entreprise.

La politique MFA doit être adaptée au niveau de risque. Pour les comptes administrateurs et environnements sensibles, MFA/WebAuthn doit être exigé.

4. Autorisation

Le contrôle d'accès combine :

- identité active ;
- rôle RBAC ;
- appartenance au workspace ;
- permissions métier ;
- binding du dataset ;
- politiques row/column ;
- audit des refus.

Les rôles principaux sont owner, admin, data scientist, analyst et viewer.

5. Confidentialité des données

Les datasets et transformations sont gouvernés par workspace. Les providers IA externes sont soumis aux politiques de confidentialité/éligibilité. Les secrets de connecteurs ne sont pas renvoyés dans les payloads utilisateurs.

6. Secrets et KMS

Le produit supporte chiffrement applicatif/enveloppe et intégrations :

- HashiCorp Vault Transit ;
- AWS KMS ;
- Google Cloud KMS ;
- Azure Key Vault.

Les secrets réels ne doivent jamais être inclus dans Git, le ZIP de release ou les images.

7. Sécurité HTTP

Contrôles présents :

- CORS explicite ;
- X-Content-Type-Options: nosniff ;
- X-Frame-Options: DENY ;
- Referrer-Policy: no-referrer ;
- Permissions-Policy restrictive ;
- HSTS sur requêtes HTTPS ;
- CSP frontend ;
- docs OpenAPI désactivées par défaut ;
- réponses d'authentification Cache-Control: no-store.

8. Uploads et contenus non fiables

Les uploads sont soumis aux limites configurées et peuvent être intégrés à un antivirus ClamAV selon le profil. En production sensible, le mode antivirus requis est recommandé.

Les formats importés doivent rester traités comme non fiables : aucune macro ou code embarqué ne doit être exécuté par le pipeline d'import.

9. Notebook sandbox

Le code utilisateur Python/R est exécuté dans un sandbox séparé. Les appels exec/eval nécessaires à l'interprétation des cellules sont limités à ce runtime dédié et ne sont pas utilisés par le control plane API.

La sécurité réelle dépend aussi de l'isolation container/runtime, des capabilities, du filesystem, du réseau et des limites de ressources de la cible.

10. Kubernetes

Les politiques de déploiement exigent notamment :

- non-root ;
- pas d'escalade de privilège ;
- seccomp RuntimeDefault ;
- suppression de toutes les capabilities Linux ;
- ressources requests/limits ;
- policy ConfTest/Rego valide.

Le bug CI rego_unsafe_var_error signalé sur capabilities.drop[_] a été corrigé par une règle Rego sûre utilisant un helper avec variable indexée dans un contexte positif.

11. Supply chain

La CI sécurité doit exécuter :

1. repository hygiene ;
2. compatibility check des pins ;
3. installation Python + pip check ;
4. pip-audit ;

5. installation frontend + typecheck/build ;
6. npm audit --audit-level=high ;
7. Trivy filesystem et images ;
8. Helm render + Conftest ;
9. tests/gates de sécurité ;
10. SBOM, provenance et digest de release.

Le conflit Docker signalé entre boto3==1.40.40 et redshift-connector==2.1.16 est corrigé en v2.81 avec boto3==1.42.22, compatible avec le plancher requis par le connecteur Redshift.

12. Audit statique v2.81

Le script scripts/security_hardening_audit.py exécute 19 contrôles reproductibles. Résultat de la branche v2.81 : **19/19 PASS**.

Il vérifie notamment : auth obligatoire, blocage local_dev en production, docs API, CORS, bootstrap, hash mot de passe, throttling, révocation/rotation, stockage navigateur, headers, CSP, politiques Kubernetes, règle Rego, confinement du code dynamique et absence de .env dans la release.

13. Validation fonctionnelle locale

- 660 tests collectés ;
- 659 passés ;
- 1 ignoré ;
- 0 échec ;
- Security Documentation Acceptance : 11/11 ;
- Security Hardening Audit : 19/19 ;
- Repository Hygiene : OK ;
- Dependency Compatibility : OK ;
- Production Baseline : OK.

14. Échecs CI remontés et statut

- **Repository hygiene** - cause : fichiers historiques résiduels à la racine d'un dossier réutilisé. Correction : release propre + script cleanup-legacy-root.ps1.
- **TypeScript AutoML** - cause : union de tâches trop restrictive. Correction : forecasting et anomaly_detection ajoutés au contrat.
- **current undefined** - cause : scope incorrect dans AIProviderControlCenter. Correction : calcul remplacé dans le callback d'état approprié.
- **unknown -> ReactNode** - cause : valeurs assistant non normalisées. Correction : conversion explicite avant rendu React.
- **Rego unsafe var** - cause : index _ sous négation. Correction : helper Rego sûr utilisant une variable indexée dans un contexte positif.
- **Docker pip resolution** - cause : pin boto3 incompatible avec Redshift. Correction : boto3==1.42.22 + gate de compatibilité.

Le typecheck frontend complet, Conftest et les scans réseau/dépendances doivent encore être reconfirmés par la CI sur l'artefact exact si la toolchain/les registries requis ne sont pas disponibles localement.

15. Risques résiduels

- **R-EXT-PENTEST - Haute.** Risque : vulnérabilité applicative non détectée automatiquement. Action : pentest indépendant avant production stable.
- **R-BROWSER-XSS - Moyenne.** Risque : les tokens Bearer restent accessibles au JavaScript pendant la session. Action : CSP stricte, audits XSS, dépendances propres et étude d'un cookie HttpOnly si le modèle de déploiement l'exige.
- **R-TARGET-TLS-IAM - Haute.** Risque : mauvaise configuration TLS/IAM/KMS/firewall sur la cible. Action : revue d'infrastructure et tests de la cible réelle.
- **R-SUPPLY-CHAIN-LIVE - Haute.** Risque : vulnérabilité nouvelle dans une dépendance ou une image. Action : pip-audit, npm audit et Trivy sur le build exact.
- **R-UAT - Moyenne.** Risque : permissions ou parcours métier incorrects. Action : UAT signée par les utilisateurs métier.

16. Critères No-Go

La production stable est refusée si l'un des cas suivants subsiste :

- High/Critical exploitable non corrigée/non acceptée formellement ;
- auth anonyme possible en production ;
- secret d'exemple actif ;
- CORS wildcard ;
- policy Kubernetes en échec ;
- pip check, typecheck ou build en échec ;
- backup non restaurable ;
- pentest avec finding critique ouvert ;
- migration non validée ;
- TLS ou IAM non validé sur cible.

17. Références de sécurité vérifiées

Références consultées pour le gel v2.81 (22 septembre 2026) :

- NIST SP 800-63B, Authentication and Authenticator Management : <https://pages.nist.gov/800-63-4/sp800-63b.html>
 - minimum de 15 caractères pour un mot de passe utilisé comme facteur unique ;
 - prise en charge recommandée d'au moins 64 caractères ;
 - blocage/rate limiting et stockage salé/hashé résistant aux attaques hors ligne.
- OWASP Password Storage Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
 - script acceptable lorsque Argon2id n'est pas disponible ; profil minimal recommandé $N=2^{17}$, $r=8$, $p=1$.
- OWASP ASVS 5.0 : <https://owasp.org/projects/asvs>

- référentiel de vérification des contrôles d'authentification, session, autorisation, API, cryptographie, configuration et protection des données.
- OWASP Cryptographic Storage Cheat Sheet :
https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
 - privilégier des systèmes dédiés de gestion de clés/secrets (KMS, Key Vault, HSM, Vault) lorsque disponibles.
- CIS Benchmarks applicables aux hôtes, conteneurs et Kubernetes selon l'environnement cible.

Ces références constituent un cadre de durcissement ; elles ne remplacent pas un pentest sur l'instance réellement déployée.